

Frontend — Documentation

Choix techniques

Le frontend d'ETNAir est une **Single Page Application** développée avec **Vue 3** dans sa syntaxe Composition API (`<script setup>`). Le bundler retenu est **Vite**, qui offre un démarrage quasi instantané en développement grâce à l'utilisation native des modules ES, ainsi qu'un Hot Module Replacement très réactif.

Pour la gestion de l'état global, le projet utilise **Pinia**, qui remplace Vuex dans l'écosystème Vue moderne. Le routing client est assuré par **Vue Router** en mode history. Les appels HTTP passent par **Axios** avec une instance centralisée et des intercepteurs pour gérer automatiquement le token JWT. La carte interactive est implémentée avec **Leaflet** et son binding Vue (`@vue-leaflet/vue-leaflet`), et certaines animations utilisent **Lottie** pour des effets vectoriels légers. Enfin, les tests end-to-end sont écrits avec **Cypress**.

Organisation du code source

Le code se trouve sous `frontend/src/`. Le point d'entrée est `main.js`, qui initialise Pinia et le router avant de monter l'application sur le DOM. Le composant racine `App.vue` définit le layout global avec la barre de navigation, le pied de page, et un composant `RouterView` qui affiche la vue correspondant à la route active.

Le dossier `views/` contient les onze pages principales de l'application : la page d'accueil avec son carousel et ses sections marketing, la liste des annonces avec ses filtres, la fiche détail d'une annonce, le tableau de bord utilisateur, les formulaires de création et d'édition d'annonces, la page favoris, les écrans de connexion et d'inscription, ainsi que les pages institutionnelles "Comment réserver" et "Qui sommes-nous".

Les composants réutilisables sont rangés dans `components/`. On y trouve la navbar et le footer bien sûr, mais aussi des éléments transversaux comme `ListingCard` pour afficher une annonce sous forme de carte, `SkeletonCard` pour le placeholder de chargement, `MapView` qui encapsule la carte Leaflet avec son système de clustering par ville, ou encore `ReviewSection` qui gère l'affichage et la création d'avis. Plusieurs composants servent uniquement à l'animation : `PageVolet` pilote la transition entre les pages, `LetterReveal` anime les titres lettre par lettre, et `CountUp` produit des compteurs qui s'incrémentent visuellement.

Les stores Pinia sont rassemblés dans `stores/`. Chacun a une responsabilité claire : `auth.js` gère l'authentification (login, register, logout) avec une expiration automatique du token au bout d'une heure, `theme.js` bascule entre mode clair et mode sombre, `favorites.js` maintient en mémoire les IDs des favoris et leur liste complète, `profilePic.js` permet à chaque utilisateur de stocker sa photo de profil en base64, et `pageTransition.js` synchronise les phases du volet d'animation avec le router.

Routing et protection des routes

Le fichier `router/index.js` déclare l'ensemble des routes de l'application. Certaines sont totalement publiques : l'accueil, la liste des annonces, la fiche détail, ainsi que les pages "Comment réserver" et "Qui sommes-nous". D'autres nécessitent une authentification : le dashboard et la page favoris portent l'attribut `meta.auth: true`, ce qui déclenche une redirection vers `/login` si l'utilisateur n'est pas connecté. Les pages d'inscription et de connexion portent à l'inverse l'attribut `meta.guest: true` : si un utilisateur déjà connecté tente d'y accéder, il est renvoyé vers l'accueil.

La création et l'édition d'annonces (`/annonces/create` et `/annonces/:id/edit`) demandent en plus d'être connecté en tant que propriétaire. Cette vérification se fait au niveau du composant, en utilisant la propriété `computed` `auth.isOwner` du store.

Le router intègre également la gestion du volet d'animation. Dans `beforeEach`, la fonction `coverPage()` est appelée pour déclencher le panneau qui glisse, puis `uncoverPage()` est invoquée dans `afterEach` pour le faire disparaître. Ce mécanisme garantit que l'animation est parfaitement synchronisée avec le chargement de la nouvelle vue.

État global avec Pinia

Le store d'authentification est le plus complexe. À l'initialisation, il lit le token éventuellement présent dans le `localStorage` et vérifie sa date d'expiration. Si le token est encore valide, l'utilisateur est considéré comme connecté. Si l'expiration est dépassée, le store nettoie tout et l'utilisateur est traité comme déconnecté. Une fois logué, un `setTimeout` est programmé pour appeler automatiquement `logout()` au moment précis de l'expiration, ce qui évite à l'utilisateur de rester connecté indéfiniment.

Le store de thème est plus simple : il maintient un booléen `dark`, persiste sa valeur dans le `localStorage`, et propage le changement en ajoutant ou retirant l'attribut `data-theme="dark"` sur la balise `<html>`. Toutes les variables CSS sont définies en fonction de cet attribut, ce qui rend la bascule entre les deux modes instantanée.

Le store des favoris maintient un `Set` des IDs favoris (structure idéale pour des `.has(id)` performants), et propose des actions `toggle(id)`, `load()` et `reset()`. Le store de photo de profil est un peu particulier : il s'agit en réalité d'un composable `useProfilePic(userId)`, ce qui permet d'afficher la bonne photo selon l'utilisateur ciblé (l'utilisateur courant dans la navbar, ou un autre utilisateur dans une section d'avis).

Design system

Le fichier `src/assets/main.css` centralise toutes les variables CSS du projet. En mode clair, les couleurs principales sont déclarées sous `:root` : la couleur primaire est un bleu `#0458a0`, le fond est gris très clair `#f9fafb`, le texte est gris très foncé `#111827`. En mode sombre, ces mêmes variables sont redéfinies sous `[data-theme="dark"]` : la couleur primaire devient blanche, le fond passe à `#18181b` (zinc-900), et plusieurs niveaux de gris sont introduits pour donner de la profondeur visuelle (`--white` pour les cartes, `--surface-2` pour les états hover, `--surface-3` pour les bordures actives).

Les animations apportent une vraie identité au site. Le volet de page glisse horizontalement à chaque navigation, ce qui masque le rechargement du contenu et donne une sensation de fluidité proche d'une vraie application. Les titres importants utilisent `LetterReveal` qui anime chaque caractère avec un léger décalage et un effet de rotation 3D subtil. Les compteurs animés sur les statistiques (`CountUp`) ajoutent une touche dynamique. Au scroll, certains éléments apparaissent grâce à la directive `v-reveal` qui utilise un `IntersectionObserver`. Enfin, chaque bouton génère une onde au clic grâce à un gestionnaire global défini dans `App.vue`.

Chaque vue importe son propre fichier CSS dédié situé dans `src/assets/css/`. Cela permet de garder les composants `.vue` lisibles et de séparer clairement la logique du style.

Communication avec l'API

Le fichier `services/api.js` contient l'instance Axios partagée. Sa `baseUrl` est configurée à `/api`, ce qui permet à Vite de proxifier les requêtes vers le backend en développement (variable d'environnement `API_URL`, qui vaut `http://api:3000` dans le contexte Docker). Un intercepteur de requête ajoute automatiquement le header `Authorization: Bearer <token>` si un token est présent dans le `localStorage`. Un intercepteur de réponse gère le cas où le serveur renvoie un 401 : dans ce cas, le token est purgé et l'utilisateur est redirigé vers `/login`.

Les différents endpoints sont regroupés par domaine dans des objets nommés : `authService` pour login et register, `listingService` pour les annonces (getAll, getOne, create, update, remove), `userService` pour les utilisateurs, et `favoriteService` pour les favoris.

Tests end-to-end avec Cypress

Le dossier `cypress/` contient quatre fichiers de spécifications qui couvrent les parcours utilisateur critiques. Le fichier `auth.cy.js` teste l'inscription et la connexion avec des identifiants valides et invalides, la déconnexion, ainsi que la protection des routes privées. Le fichier `annonces.cy.js` vérifie l'affichage de la liste, le filtre par ville, les différentes vues (grille, liste, carte), et la navigation vers une fiche détail. Le fichier `dashboard.cy.js` couvre l'accès au tableau de bord, l'affichage des onglets, la liste des annonces publiées et la suppression. Enfin `favoris.cy.js` teste l'ajout et le retrait des favoris, ainsi que l'affichage de la page dédiée.

Les appels API sont systématiquement interceptés avec `cy.intercept()` et alimentés par des fixtures JSON, ce qui rend les tests indépendants du backend. Cela permet de les exécuter en CI sans avoir besoin de démarrer une vraie base de données.

Pour lancer Cypress en local, il suffit d'exécuter `npm run cy:open` (mode interactif avec UI) ou `npm run cy:run` (mode headless utilisé en CI).

Configuration Vite

Le fichier `vite.config.js` définit le proxy `/api` qui redirige toutes les requêtes commençant par ce préfixe vers le backend. La cible est lue depuis la variable d'environnement `API_URL`, ce qui permet de pointer vers `http://localhost:3000` en développement local hors Docker, ou vers `http://api:3000` quand le frontend tourne dans un conteneur Compose. La règle `rewrite` retire le préfixe `/api` avant de relayer la requête, puisque le backend Express expose ses routes directement à la racine.

Conventions de code

Tous les composants utilisent la **Composition API** avec `<script setup>`, qui offre une syntaxe plus concise que l'Options API. Les noms de composants suivent la convention PascalCase, les noms de stores commencent par `use` (par exemple `useAuthStore`) et exposent des refs et des actions sans utiliser le modèle Options.

Pour les couleurs dans les CSS, l'approche standard est d'utiliser les variables comme `var(--primary)` ou `var(--text)`. Cependant, pour certains éléments où le contraste est critique (par exemple un texte sur fond coloré qui change en dark mode), il vaut mieux utiliser directement des sélecteurs `[data-theme="dark"]` avec des couleurs hardcodées, afin d'éviter les problèmes de lisibilité.